

Command history functionality added to 6809/6800 and FLEX: FLEX9.

 flexonsbd.blogspot.com/2018/09/flex9.html

Saturday, September 15, 2018

```
>FLEX
6809 FLEX V3.01
DATE (MM,DD,YY)? 9,15,18
+++CAT
Catalog of Drive Number: 1
DiskName: FLEX9SYS, DiskNo: 0
Name      Type Size P
-----
ZAP772   .TXT   24  SET_LIN2.TXT   33  SET_LIN2.CMD    5  ZAP77   .TXT   24
SETL     .TXT   38  SETL     .CMD    5  ZAP7721 .TXT   24  ZAP77   .TXT   24
CVTAB    .TXT    8  CVTAB    .CMD    2
Sectors left = 2341
```

Command history function added to FLEX9

We've added a command history (history) function to FLEX9.

I think FLEX is easy to use, with a good selection of languages and utilities despite being a simple OS, but of course, there are various things I'm dissatisfied with.

One of them is the lack of a command history function. When creating a program, it's painful to repeatedly enter the same commands. This is especially true when using assembly language, where the frequency of repetition is particularly high, which can be stressful.

To reduce the amount of repetitive input, I created and use MAKEA.CMD and MAKEC.CMD, which are used when developing programs in assembly language or MicroC.

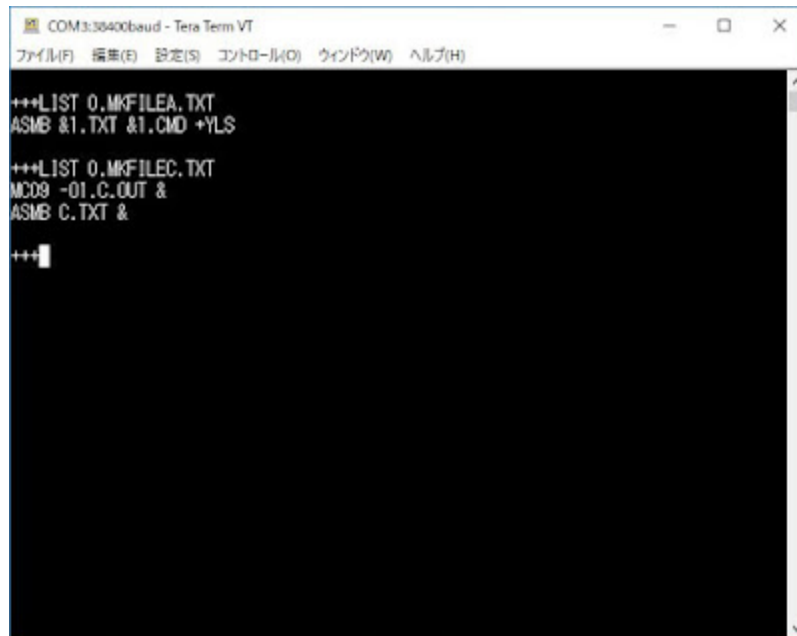
```
COM3:38400baud - Tera Term VT
ファイル(F) 編集(E) 設定(S) コントロール(O) ウィンドウ(W) ヘルプ(H)
>FLEX
8809 FLEX V3.01
DATE (MM,DD,YY)? 9,15,18
+++CAT
Catalog of Drive Number: 1
DiskName: FLEX9SYS, DiskNo: 0
Name  Type Size P
-----
ZAP772 .TXT 24 SET_LIN2.TXT 33 SET_LIN2.CMD 5 ZAP77 .TXT 24
SETL .TXT 38 SETL .CMD 5 ZAP7721 .TXT 24 ZAP77 .TXT 24
CVTAB .TXT 8 CVTAB .CMD 2
Sectors left = 2841
+++MAKEA CVTAB
0 ERROR(S) DETECTED
LAST ASSEMBLED ADDRESS: C204
+++
```

Example of using MAKEA.CMD

```
COM3:38400baud - Tera Term VT
ファイル(F) 編集(E) 設定(S) コントロール(O) ウィンドウ(W) ヘルプ(H)
+++MAKEC MOT2CMD
->MC09 -01.C.OUT MOT2CMD.C
MOT2CMD.C:
#include "STD10.TXT"
End of inclusion.
#include "ALLOC.TXT"
End of inclusion.
#include "STRING.TXT"
End of inclusion.
#include "FILE10.TXT"
End of inclusion.
Compiled 777 lines.
Total internal labels : 282.
Total global variables : 43 bytes.
->ASMB C.TXT MOT2CMD.CMD +YLS
0 ERROR(S) DETECTED
LAST ASSEMBLED ADDRESS: 1CD3
+++
```

Example of using MAKEC.CMD

Both methods assume that the system files (ASMB.CMD, MC09.CMD, etc.) are on drive 0 and the target files are on drive 1, but they are still somewhat useful. For reference, the contents of the respective makefiles (MKFILEA.TXT, MKFILEC.TXT) are shown below. The target file name is read and executed at the positions of &1 and &.

A screenshot of a Tera Term VT window titled 'COM3:38400baud - Tera Term VT'. The window has a menu bar with 'ファイル(F)', '編集(E)', '設定(S)', 'コントロール(C)', 'ウィンドウ(W)', and 'ヘルプ(H)'. The main text area is black with white text showing assembly code. The code includes two '+++LIST' commands followed by file names, and two 'ASMB' commands with file names and a '+YLS' flag. The first 'ASMB' command is followed by '&1.TXT &1.CMD +YLS'. The second 'ASMB' command is followed by '&1.CMD +YLS'. The third 'ASMB' command is followed by '&1.CMD +YLS'. The fourth 'ASMB' command is followed by '&1.CMD +YLS'. The fifth 'ASMB' command is followed by '&1.CMD +YLS'. The sixth 'ASMB' command is followed by '&1.CMD +YLS'. The seventh 'ASMB' command is followed by '&1.CMD +YLS'. The eighth 'ASMB' command is followed by '&1.CMD +YLS'. The ninth 'ASMB' command is followed by '&1.CMD +YLS'. The tenth 'ASMB' command is followed by '&1.CMD +YLS'. The eleventh 'ASMB' command is followed by '&1.CMD +YLS'. The twelfth 'ASMB' command is followed by '&1.CMD +YLS'. The thirteenth 'ASMB' command is followed by '&1.CMD +YLS'. The fourteenth 'ASMB' command is followed by '&1.CMD +YLS'. The fifteenth 'ASMB' command is followed by '&1.CMD +YLS'. The sixteenth 'ASMB' command is followed by '&1.CMD +YLS'. The seventeenth 'ASMB' command is followed by '&1.CMD +YLS'. The eighteenth 'ASMB' command is followed by '&1.CMD +YLS'. The nineteenth 'ASMB' command is followed by '&1.CMD +YLS'. The twentieth 'ASMB' command is followed by '&1.CMD +YLS'. The twenty-first 'ASMB' command is followed by '&1.CMD +YLS'. The twenty-second 'ASMB' command is followed by '&1.CMD +YLS'. The twenty-third 'ASMB' command is followed by '&1.CMD +YLS'. The twenty-fourth 'ASMB' command is followed by '&1.CMD +YLS'. The twenty-fifth 'ASMB' command is followed by '&1.CMD +YLS'. The twenty-sixth 'ASMB' command is followed by '&1.CMD +YLS'. The twenty-seventh 'ASMB' command is followed by '&1.CMD +YLS'. The twenty-eighth 'ASMB' command is followed by '&1.CMD +YLS'. The twenty-ninth 'ASMB' command is followed by '&1.CMD +YLS'. The thirtieth 'ASMB' command is followed by '&1.CMD +YLS'. The thirty-first 'ASMB' command is followed by '&1.CMD +YLS'. The thirty-second 'ASMB' command is followed by '&1.CMD +YLS'. The thirty-third 'ASMB' command is followed by '&1.CMD +YLS'. The thirty-fourth 'ASMB' command is followed by '&1.CMD +YLS'. The thirty-fifth 'ASMB' command is followed by '&1.CMD +YLS'. The thirty-sixth 'ASMB' command is followed by '&1.CMD +YLS'. The thirty-seventh 'ASMB' command is followed by '&1.CMD +YLS'. The thirty-eighth 'ASMB' command is followed by '&1.CMD +YLS'. The thirty-ninth 'ASMB' command is followed by '&1.CMD +YLS'. The fortieth 'ASMB' command is followed by '&1.CMD +YLS'. The forty-first 'ASMB' command is followed by '&1.CMD +YLS'. The forty-second 'ASMB' command is followed by '&1.CMD +YLS'. The forty-third 'ASMB' command is followed by '&1.CMD +YLS'. The forty-fourth 'ASMB' command is followed by '&1.CMD +YLS'. The forty-fifth 'ASMB' command is followed by '&1.CMD +YLS'. The forty-sixth 'ASMB' command is followed by '&1.CMD +YLS'. The forty-seventh 'ASMB' command is followed by '&1.CMD +YLS'. The forty-eighth 'ASMB' command is followed by '&1.CMD +YLS'. The forty-ninth 'ASMB' command is followed by '&1.CMD +YLS'. The fiftieth 'ASMB' command is followed by '&1.CMD +YLS'. The fifty-first 'ASMB' command is followed by '&1.CMD +YLS'. The fifty-second 'ASMB' command is followed by '&1.CMD +YLS'. The fifty-third 'ASMB' command is followed by '&1.CMD +YLS'. The fifty-fourth 'ASMB' command is followed by '&1.CMD +YLS'. The fifty-fifth 'ASMB' command is followed by '&1.CMD +YLS'. The fifty-sixth 'ASMB' command is followed by '&1.CMD +YLS'. The fifty-seventh 'ASMB' command is followed by '&1.CMD +YLS'. The fifty-eighth 'ASMB' command is followed by '&1.CMD +YLS'. The fifty-ninth 'ASMB' command is followed by '&1.CMD +YLS'. The sixtieth 'ASMB' command is followed by '&1.CMD +YLS'. The sixty-first 'ASMB' command is followed by '&1.CMD +YLS'. The sixty-second 'ASMB' command is followed by '&1.CMD +YLS'. The sixty-third 'ASMB' command is followed by '&1.CMD +YLS'. The sixty-fourth 'ASMB' command is followed by '&1.CMD +YLS'. The sixty-fifth 'ASMB' command is followed by '&1.CMD +YLS'. The sixty-sixth 'ASMB' command is followed by '&1.CMD +YLS'. The sixty-seventh 'ASMB' command is followed by '&1.CMD +YLS'. The sixty-eighth 'ASMB' command is followed by '&1.CMD +YLS'. The sixty-ninth 'ASMB' command is followed by '&1.CMD +YLS'. The seventieth 'ASMB' command is followed by '&1.CMD +YLS'. The seventy-first 'ASMB' command is followed by '&1.CMD +YLS'. The seventy-second 'ASMB' command is followed by '&1.CMD +YLS'. The seventy-third 'ASMB' command is followed by '&1.CMD +YLS'. The seventy-fourth 'ASMB' command is followed by '&1.CMD +YLS'. The seventy-fifth 'ASMB' command is followed by '&1.CMD +YLS'. The seventy-sixth 'ASMB' command is followed by '&1.CMD +YLS'. The seventy-seventh 'ASMB' command is followed by '&1.CMD +YLS'. The seventy-eighth 'ASMB' command is followed by '&1.CMD +YLS'. The seventy-ninth 'ASMB' command is followed by '&1.CMD +YLS'. The eightieth 'ASMB' command is followed by '&1.CMD +YLS'. The eighty-first 'ASMB' command is followed by '&1.CMD +YLS'. The eighty-second 'ASMB' command is followed by '&1.CMD +YLS'. The eighty-third 'ASMB' command is followed by '&1.CMD +YLS'. The eighty-fourth 'ASMB' command is followed by '&1.CMD +YLS'. The eighty-fifth 'ASMB' command is followed by '&1.CMD +YLS'. The eighty-sixth 'ASMB' command is followed by '&1.CMD +YLS'. The eighty-seventh 'ASMB' command is followed by '&1.CMD +YLS'. The eighty-eighth 'ASMB' command is followed by '&1.CMD +YLS'. The eighty-ninth 'ASMB' command is followed by '&1.CMD +YLS'. The ninetieth 'ASMB' command is followed by '&1.CMD +YLS'. The hundredth 'ASMB' command is followed by '&1.CMD +YLS'.

```
+++LIST 0,MKFILEA.TXT  
ASMB &1.TXT &1.CMD +YLS  
  
+++LIST 0,MKFILEC.TXT  
MCD9 -01.C.OUT &  
ASMB C.TXT &  
  
+++
```

Contents of the makeup file

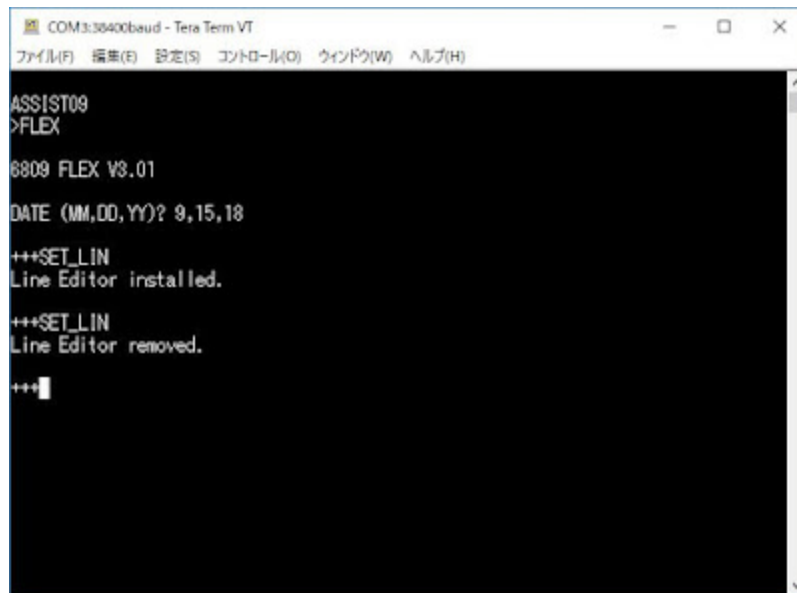
However, a history function is absolutely necessary in all but these specific cases.

The history function is implemented by hooking into FLEX command input.

When reading the FLEX source code, the command input routine starts from \$CE2B, but it only has editing functions like backspace (BS). So, we hook into this and replace it with our own command input routine.

To place our custom input routine, we reduce the Memory End value of \$CC2B,2C by the size of the input routine to create an area not used by the system, and place it there.

This is how we created SET_LIN.CMD.



Keep SET_LIN.CMD running in the background.

When the system is waiting for command input, typing SET_LIN will make it run in the background, and typing SET_LIN again will cancel it.

The editing functions include:

- Ctrl-A to recall history (history function)

 - recalls the part of the command that was entered from that point onward

(completion function)

- Ctrl-E to delete everything from the cursor position onward.

- ↑ to jump to the beginning of the command string.

- ↓ to jump to the end of the command string.

- ← to move one character to the left.

- to move one character to the right.

- BS to delete the character to the left of the cursor position.

- DEL to delete the character at the cursor position.

- CR to finish editing. The input string is processed as a command.

The

history function is insufficient for those of us accustomed to Windows, as it can only recall the most recent command, but it still significantly reduces stress.

Regarding production

Actually, when I was using FLEX9 and OS-9Level2 on my FM77AV40SX, I created and used

a simple SET_LIN.CMD to implement OS-9's history function in FLEX. (This version lacked the Ctrl-A completion and ↑/↓ arrow keys of the above function.) Even this was easier to use than OS-9's ultra-simple history function.

Since I implemented the function using the FM77's BIOS and Subsystem, the arrow keys were naturally usable, so when creating this program, I really wanted to implement arrow key processing, but this was the most difficult part.

TeraTerm uses escape sequences for screen control, but when trying to handle the arrow keys, you have to deal with 3 characters like \$1B, \$5B, \$41 (↑ key). Also, all the keys on the numeric keypad are 3 characters, so I had to handle those in the program as well. (While it's understandable that the arrow keys are treated as 3 characters, couldn't the others simply be treated as 1 character?)

The program size has also more than doubled from around 430 bytes for the FM77 to about 1K bytes because the BIOS and Subsystem editing functions also need to be implemented in the program. I

plan to continue using it for a while longer to confirm that there are no problems before releasing it.